

GhostBusters: Read-Only Technical Audit

Prepared for AI competition pitching • Audit scope: source, configuration, tests, API routes, frontend, integrations, fixtures and infrastructure files • 30 July 2026

Bottom line. GhostBusters is a safety-first cloud-cost investigation and Terraform PR-review application. Its strongest complete demonstration is a fixture-backed review workflow that collects evidence, computes alternatives/confidence/policy, requires human review, and produces a simulated PR. Real GitHub/AWS/Jira paths are implemented but optional and credential-dependent. It never applies Terraform, merges PRs, deploys cloud resources, or auto-approves changes.

1. Current project truth

The application is a FastAPI web app with a static browser UI. A user may start a fixture scenario through `POST /api/runs`, submit a high-level goal through `/api/goals`, run a Cloud Hunt, or receive an allowed GitHub pull-request webhook at `POST /webhooks/github`.

- 1. Terraform fixture plan JSON or GitHub PR diff is parsed (`integrations/terraform_parser.py`, `integrations/terraform_runner.py`).
- 2. `core/planner.py` selects registered evidence tools; optional AI may propose bounded next tools.
- 3. `core/investigator.py` collects evidence; `core/reasoning_engine.py` builds alternatives, conflicts, confidence, verifier results and policy result.
- 4. Recommendation-ready cases stop for permissioned human review at `/api/runs/{run_id}/review`.
- 5. Approval creates *simulated* PR data by default (`core/mock_pr.py`). With explicit GitHub configuration, `integrations/remediation_pr_service.py` can create one restricted remediation branch/file commit/PR.

Nothing is approved by AI, merged, deployed, or applied automatically. The non-demo connected-goal path (`WorkflowService.start_connected_goal`) is read-only and often ends as `needs_more_evidence` or `abstained`, rather than producing a full remediation.

2. Feature implementation matrix

Status key: Implemented = executable code path; Partial = bounded/incomplete; Mock = fixture/simulated; Absent = not found.

Feature	Status	How it works / evidence	Real or mocked	Demo-ready
User authentication	Implemented	<code>app/auth.py</code> ; <code>/api/auth/*</code> ; session, CSRF and RBAC.	Real local/Postgres-capable	Yes
Repository connection / indexing	Partial	GitHub App callback/config; lists repos, PR files and CODEOWNERS in <code>integrations/github_*.py</code> . No general codebase index.	Real when configured	Conditional
Terraform parsing / plan generation	Implemented / Partial	JSON parser and PR diff parser; <code>TerraformRunner</code> permits <code>validate/fmt-check/show-json</code> , never <code>plan</code> or <code>apply</code> .	Fixture or real PR input	Yes / no plan generation
Infrastructure graph	Absent	No graph model, store or visualizer found.	—	No
AWS discovery / Pricing / CloudWatch	Implemented	<code>RealAWSCloudAdapter</code> : STS, EC2/EBS/EIP discovery, CloudWatch CPU, Pricing API.	Real optional; fixture alternative	Conditional
Cost Explorer	Absent	No AWS Cost Explorer client/call found.	—	No
Jira / GitHub	Implemented	<code>jira_client.py</code> , <code>github_client.py</code> , signed webhook and GitHub App.	Real optional	Conditional

Linear / GitLab	Absent	No integration/configuration found.	—	No
Goal creation, validation, rewriting	Implemented / Partial	<code>/api/goals</code> , <code>/api/goals/validate</code> ; deterministic clarification and <code>suggested_goal</code> , but no automatic rewrite persistence.	Real app state; AI optional	Yes
Goal decomposition / multi-agent orchestration	Partial	Allowlisted <code>GeminiGoalPlan</code> and one <code>AIPlanner</code> loop; no independent specialist-agent bus.	Optional Gemini/Groq/mock	Conditional
Dynamic tool selection	Partial	<code>AgentNextAction</code> can select registered read-only tool; <code>ai_plan_validator.py</code> restricts it.	AI optional; deterministic authority	Conditional
Recommendations / ranking / savings / confidence	Implemented	<code>alternative_generator.py</code> , <code>reasoning_engine.py</code> , <code>confidence.py</code> .	Fixture pricing in demo; live optional	Yes, labelled
Risk / dependencies / policy / deterministic validation	Implemented	<code>conflict_detector.py</code> , <code>verifier.py</code> , <code>policy_engine.py</code> , <code>policies/ghostbusters.rego</code> .	Real local logic	Yes
AI validation	Partial	Structured Gemini/Groq outputs in <code>ai_client.py</code> ; only enabled/configured in relevant paths.	Optional	Conditional
Failure recovery / abstention	Implemented	<code>retry.py</code> ; outcomes include <code>abstained</code> , <code>keep</code> , <code>needs_more_evidence</code> , <code>blocked</code> .	Real local logic	Yes
Terraform code generation	Partial	Exact safe size-attribute replacement only in <code>RemediationPRService.create()</code> .	Real optional	Conditional
Branch, commit, PR	Implemented	<code>GitHubClient.create_branch</code> , <code>update_or_create_file</code> , <code>create_pull_request</code> .	Real only when explicitly enabled; simulated default	Simulated yes
PR review / human approval	Implemented	Webhook/PR workflow; permissioned actions, versions and idempotency in <code>app/main.py</code> .	Real app state	Yes
Automatic approval, merging, deployment	Absent	No route/API call for agent approval, GitHub merge or Terraform/cloud apply.	—	No
Decision timeline / explainability / audit	Implemented	<code>AuditEvent</code> , <code>/api/goals/{id}/events</code> , <code>activity</code> , <code>evidence</code> , <code>policy</code> , <code>confidence</code> and UI technical trace.	Real persisted/in-memory	Yes
Learning from outcomes	Partial	<code>outcome_verification.py</code> stores observations; no feedback loop retrains or changes ranking.	Manual observation	Partial

3. Autonomous-agent analysis

Assessment: bounded, safety-constrained AI-assisted investigation—not a general autonomous cloud-operations agent.

Question	Answer and source evidence
Accept a high-level goal?	Yes. <code>/api/goals/validate</code> and <code>/api/goals</code> accept text, scope, repositories, constraints and criteria.
Create a plan / select tools?	Partially. <code>AIPlanner.plan()</code> and <code>GeminiGoalPlan</code> can propose; <code>core/planner.py</code> and allowlists are authoritative.
Change plan from evidence?	Partially. The loop chooses next registered tools and can request human context; it cannot invent tools, permissions or mutations.
Handle ambiguity/request information?	Yes. Deterministic percentage-target questions and optional AI clarifications in <code>/api/goals/validate</code> ; <code>request_human_context</code> action exists.
Generate/select alternatives?	Yes. <code>alternative_generator.py</code> creates <code>downsize</code> , <code>schedule</code> , <code>keep</code> , <code>request-evidence</code> , <code>abstain/blocked</code> options; <code>_select_preferred</code> ranks them.
Recover/fail safely?	Yes. <code>RetryExecutor</code> retries transient failure and records classified, safe outcomes.
Stop on low confidence?	Yes. Default threshold is <code>0.70</code> ; Rego/Python policy blocks remediation.
Dynamic or hard-coded order?	Both: bounded dynamic tool selection within a largely hard-coded deterministic safety pipeline.

AI vs deterministic decisions?	AI: interpretation, optional validation/plan/next-tool/explanation. Deterministic: parsing, allowlists, evidence execution, alternatives, conflicts, confidence, verifier, policy, eligibility. If/else: production/destructive/data-mode gates and action preference.
--------------------------------	--

4. Actual agents

Component	Path	Responsibility / I-O	Model/tools	Live status
AIPlanner	core/ai_planner.py	Goal/scenario/resource/registry → selected tools, evidence, next actions.	Gemini, Groq or MockGemini; registered tools.	Used when configured
Agent loop	core/agent_loop.py	Thin endpoint → AIPlannerResult .	Same	Used
Goal AI planner	app/main.py	Connected goal → allowlisted capability plan.	Gemini/Groq optional	Production-configured path
Deterministic planner	core/planner.py	Terraform resource/scenario → selected tools/questions.	No model	Used
Investigator	core/investigator.py	Plan → evidence/execution records.	Registry + retry executor	Used
Verifier	core/verifier.py	Resource/evidence/alternative → findings.	No model	Used
Reasoning engine	core/reasoning_engine.py	Evidence → decision record.	Optional explanation model	Used
Cloud Hunt service	core/cloud_hunt_service.py	Inventory → candidates/cases.	No LLM	Used

No named Goal Planner, Repository Understanding Agent, Cloud Hunt Agent, Optimizer, PR Approval Agent, or LangGraph Investigator/Optimizer/Verifier classes were found. These components communicate through in-process function calls and persisted run/audit state, not agent messages.

5. Tools and external integrations

Tool/service	Purpose and evidence	Auth / errors / live status
GitHub REST/App	github_client.py , github_app.py , github_webhook.py : PR/files/reviews/repos and optional branch/file/PR writes.	Bearer token or App installation token; signed webhook; sanitized/retried reads. Live only with credentials/config.
AWS	cloud_adapters.py : STS, EC2, EBS, EIP, CloudWatch, Pricing.	Default/assumed role with external ID; read-only, returns unavailable evidence safely. Live optional.
Jira	jira_client.py , jira_context.py : issue/project context.	Base URL/email/API token; failures safe. Live optional.
Terraform CLI	terraform_runner.py : version/fmt-check/validate/show JSON.	System executable; disabled by default; no apply or plan.
OPA/Conftest	conftest_policy.py , policies/ghostbusters.rego .	Optional CLI with deterministic Python fallback.
PostgreSQL / Redis	docker-compose.yml , schema and stores.	Optional locally; production validation requires configuration.
Gemini / Groq	ai_client.py , google-genai dependency.	API keys, schemas and fallbacks. Groq client code exists but its package is not listed in requirements.txt .
Azure/GCP	AzureCloudAdapter , GCPCloudAdapter .	Fixture-only; no real cloud SDK integration.
Linear/GitLab/Slack/Teams/OpenAI	No implementation found.	Absent.

Conclusion: code genuinely supports at least three external integration paths—GitHub, AWS and Jira—although this audit did not validate live credentials or accounts.

6. Human-in-the-loop and safety

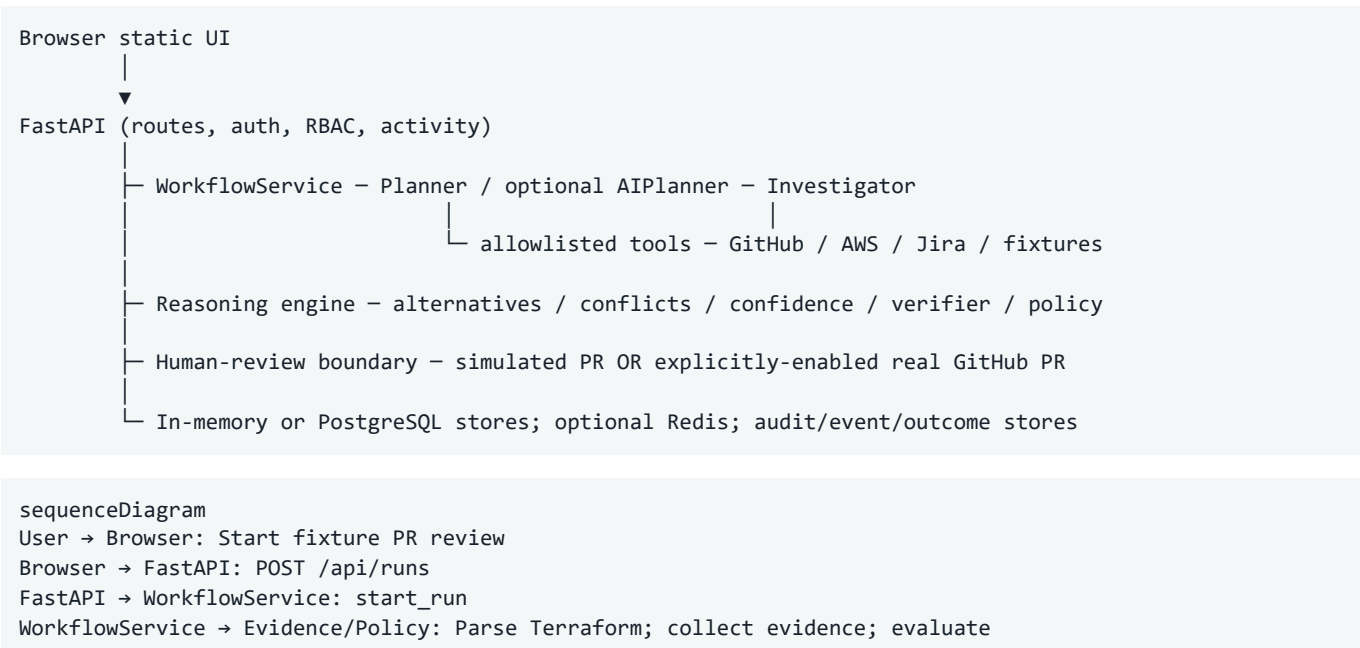
- Approval is mandatory for a remediation action: `/api/runs/{run_id}/review`, `core/human_review.py`.
- Supported human actions: approve, reject, modify, request evidence, add context, revoke approval and reopen; Cloud Hunt additionally supports waiver.
- Approval records idempotency, state version, actor snapshot and audit/activity event (`human_decision_events`, `audit_log`, `activity_events`).
- Production, destructive changes, active dependencies, missing critical evidence and confidence below `0.70` are blocked by `ghostbusters.rego` and fallback policy.
- `RemediationPRService.create()` separately rejects destructive/replacement/production/dependency/missing-evidence/ambiguous code changes.
- No AI approval, automatic merge, deployment, Terraform apply, rollback or cloud mutation exists.

7. Explainability and decision trail

The UI in `static/index.html` and `static/app.js` displays Goals, Operations/PR Reviews, Approvals, Cloud Hunt, Activity Log, Technical Audit and Demo Readiness. Goal workspace views include Outcome, Live Plan, Findings, Evidence and Technical Trace.

Layer	What is available	Evidence
Frontend	Timeline, tool attempts, evidence summaries, confidence, policy/verifier state, approval controls and failure messaging.	<code>static/index.html</code> , <code>static/app.js</code>
API	Runs, reviews, activity, goal events, outcomes, assistant answers.	<code>/api/runs</code> , <code>/api/reviews</code> , <code>/api/activity</code> , <code>/api/goals/{id}/events</code> , <code>/api/outcomes</code>
Persisted	Audit events, decision events, evidence, approvals and outcomes.	<code>db/schema.sql</code> , <code>core/audit.py</code>
Raw logs	Python logging for API, AI and retry detail; distinct from UI/audit records.	<code>app/main.py</code> , <code>core/ai_client.py</code> , <code>core/retry.py</code>

8. Architecture



```

Evidence/Policy → WorkflowService: Recommendation or safe stop
WorkflowService → Browser: Pending human review
Reviewer → FastAPI: Approve
FastAPI → WorkflowService: review_run
WorkflowService → Browser: Simulated PR by default
WorkflowService → GitHub: Optional real branch/file/PR only when explicitly configured

```

9. Live demo readiness

1. Create and activate the expected `.venv`; install `requirements.txt`.
2. Set `DEMO_MODE_ENABLED=true`; start `uvicorn app.main:app --reload`.
3. Open `http://127.0.0.1:8000`; register; open Demo Readiness.
4. Run the `safe` scenario; show Terraform parsing, labelled fixture evidence, policy, confidence and recommendation.
5. Approve it; describe the resulting simulated PR truthfully.
6. Run `conflicting` and `missing_evidence`; show conflict penalty and safe escalation.
7. Close with Activity Log/Technical Audit.

Reliability. The fixture path is the strongest 5–7 minute demo, as documented in `DEMO_GUIDE.md`. Real GitHub/AWS/Jira/Gemini depend on credentials, permissions, network and preflight. `run_demo.ps1` assumes an existing virtual environment. During this audit, tests could not be run because neither `pytest` command nor the Python `pytest` module was installed in the current environment.

10. Measurable results

Value	Meaning / source	Provenance / presentation safety
\$70/month; \$840/year	Safe-scenario estimate; <code>fixtures/scenarios/safe.json</code> , <code>tests/test_alternatives.py</code> .	Fixture estimate. Safe only when labelled demo/estimate.
18% average; 31% peak CPU; 14 days	Safe scenario utilization.	Fixture data. Label it.
10 inventory resources	<code>fixtures/cloud_inventory.json</code> : AWS 4, Azure 3, GCP 3.	Fixture data. Safe if labelled.
3 AWS families	EC2, EBS, Elastic IP in <code>RealAWSAdapter</code> .	Implemented capability, not a verified live result.
0.70	Minimum policy confidence; <code>app/settings.py</code> , Rego.	Actual default, safe.
0.45 / 0.75	Cloud Hunt candidate/high-confidence thresholds.	Actual defaults, safe.

No real savings, cost-reduction percentage, accuracy, coverage percentage, latency benchmark, or live PR-time result was found. Do not invent them.

11. Current technology stack

- Python, FastAPI, Pydantic, Uvicorn: `requirements.txt`, `app/main.py`.
- Vanilla HTML/CSS/JavaScript UI: `static/`.
- PostgreSQL 17 / Redis 7: `docker-compose.yml`; optional locally, required by production settings validation.
- boto3/AWS, google-genai/Gemini, optional Terraform/Conftest executables.
- Docker Compose and Render: `docker-compose.yml`, `render.yaml`.
- Groq code is present but its package is missing from `requirements.txt`. No OpenAI SDK/API is used.

12. Contradictions resolved

1. **Goal-driven vs PR-triggered:** both; PR-review/fixture is most complete, goals are partial read-only workflows.
2. **Human vs AI approval:** human approval mandatory; automatic AI approval absent.
3. **PR creation vs automatic merge:** optional real PR creation; no merge.
4. **Recommendation vs deployment:** recommendation/PR only; no deployment.
5. **LangGraph specialist agents:** no LangGraph or named specialist-agent architecture found.
6. **Gemini-only:** Gemini, optional Groq and mock client; no OpenAI provider.
7. **Real vs mock AWS:** both; real AWS optional, fixture demo default.
8. **Project management:** real Jira read-only path plus fixture Jira context.
9. **Fixed vs dynamic:** bounded dynamic selection inside a deterministic safety pipeline.
10. **Dashboard vs goal-driven:** dashboard-centered UI with both goals and PR-review entry points.

13. Claims for the presentation

Safe claims

- Evidence-backed, policy-checked cloud optimization recommendations with an auditable trail.
- Mandatory human approval before remediation PR generation.
- Safe blocking for production, destructive changes, dependencies, critical missing evidence and low confidence.
- Optional read-only GitHub, AWS, Jira and AI-provider integrations.

Claims requiring careful wording

- “Autonomous investigation” → say “bounded, allowlisted AI-assisted investigation.”
- “Creates PRs” → say “simulated PR in demo; optional real GitHub PR after explicit configuration and human approval.”
- “Multi-cloud” → fixtures span AWS/Azure/GCP; real discovery is AWS-only.
- “Savings” → live AWS path exists, but demo values are fixtures.

Do not claim

- Autonomous deployment, Terraform apply, automatic merging, automatic approval, rollback, Cost Explorer, Linear/GitLab/Slack/Teams, infrastructure graph, real Azure/GCP operations, verified production savings, accuracy, test coverage or multiple independent agents.

14. Judging-criteria mapping

Criterion	Strong evidence	Weakness / honest answer
B2B impact	FinOps case, RBAC, organization scope, audit and approval.	No customer validation or real savings evidence.
Autonomous reasoning	Bounded AI next-tool selection, alternatives and abstention.	Decision authority and order are mostly deterministic.
Technical architecture	FastAPI, stores, RBAC, audit, policy and integrations.	AWS-only real cloud support; external paths unverified in this audit.
Human-in-the-loop	Permissioned review, version/idempotency, policy gates.	Default remediation output is simulated.
Live demo	Safe/conflict/missing-evidence fixture flow.	Clearly label all fixture data and simulated PRs.

Likely judge question: “Does it make changes automatically?” **Honest answer:** “No. It stops at a mandatory human decision boundary. An explicitly configured route can create a narrow real GitHub remediation PR after human approval, but it never merges or applies infrastructure.”

15. Final executive summary

GhostBusters truly is a polished, safety-first cloud-cost investigation and Terraform PR-review platform. Its strongest competition differentiator is the combination of evidence-aware recommendations, abstention, policy gates, mandatory human approval and a visible audit trail.

Strongest live demonstration: the fixture-backed safe recommendation followed by conflicting and missing-evidence cases, with source-mode labels visible.

Three strongest implemented capabilities: deterministic safety evaluation; transparent decision/audit experience; optional guarded GitHub/AWS/Jira integration paths.

Three largest limitations: fixture/simulated core demo; incomplete goal-to-remediation automation; no apply/merge/deploy, Cost Explorer, infrastructure graph or real Azure/GCP support.

Presentation correction required: remove any implication of autonomous deployment, automatic approval/merge, Cost Explorer, real multi-cloud monitoring, or multiple independent agent roles.

Read-only audit. No project source, configuration or runtime data was modified while preparing the findings. This report document was generated at the user's request.